Clip to Grok: Weight Norm Clipping for Accelerated Generalization

Vladimir Volchkov[1] Aviad Rivlin[2]

[1,2]Independent Researchers

Abstract

Grokking—delayed generalization long after memorization—is controlled by weight norm dynamics. We show that a simple intervention, per-row weight norm clipping on decoder layers, accelerates grokking by 66× on 2-layer models and 18× on 8-layer models (1.6M parameters) while eliminating the need for weight decay. The method requires a few lines of code and no additional memory overhead.

We provide an analysis connecting weight norm clipping to four established frameworks: (i) Omnigrok's Goldilocks zone timescale, which collapses when norms are clamped to $w \approx w_c$; (ii) the α^L depth scaling law, which motivates our *edge initialization* strategy for deep models; (iii) sign-based optimizers' uniform ±lr updates, which combine naturally with ℓ_2 clipping to stabilize training across learning rates; and (iv) Softmax Collapse prevention, which eliminates the post-memorization logit growth that stalls standard training. Edge initialization—normalizing only embeddings, the final LayerNorm, and the output head—achieves zero failures across 300 runs on 8-layer architectures and reduces interquartile range by 61–72%.

We observe that Grokfast's gradient EMA filter, when passed through Adam's second-moment normalization, produces updates that approximate sign-based optimizers like Lion—suggesting that prior acceleration methods may have succeeded partly through implicit norm-constrained dynamics.

1 Introduction

Grokking—the phenomenon where neural networks generalize long after memorizing training data—was first documented by Power et al. [2022] on small algorithmic datasets. The delay between memorization and generalization can span tens of thousands of optimization steps, raising fundamental questions about the dynamics of neural network learning. Subsequent work has converged on a central insight: *the grokking delay is controlled by weight norm dynamics.*

Liu et al. [2023] showed that generalization concentrates in a "Goldilocks zone"—a narrow spherical shell in weight space—and that constraining weight norms to this zone nearly eliminates grokking. Lee et al. [2024] approached the same problem from the gradient perspective, amplifying slow-varying gradient components to accelerate generalization by over 50×. Prieto et al. [2025] revealed that unconstrained weight growth leads to Softmax Collapse, a floating-point failure mode that stalls generalization entirely. Each line of work arrives at the same conclusion from a different angle: *controlling weight magnitude is the key to fast generalization.*

We take the most direct path: clipping each decoder layer's weight row norms to a small constant after every optimizer step. With no additional memory overhead, this achieves a 66× speedup over baseline on 2-layer architectures and 18× on 8-layer architectures (1.6M parameters), while eliminating post-convergence instability and the need for weight decay. The method works across optimizers—Adam, Lion, and SignSGD—with Lion+Clip achieving the strongest results and notably stabilizing Lion's typically narrow learning rate tolerance. Figure 1 illustrates the effect on a single 2-layer run.

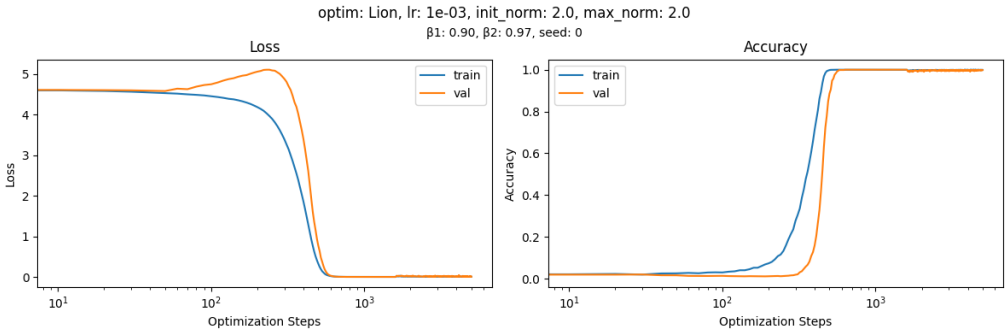


Figure 1: Single-seed demonstration on the 2-layer architecture (Lion, lr= 10^{-3}, `init_norm`= 2.0, `max_norm`= 2.0, β_1=0.9, β_2=0.97, 422k params). Training and validation converge together within $\sim$1000 steps—no grokking delay. For 8-layer models, we use `init_pattern='edge_ln'` instead (Section 4.2).

Contributions.

1. **A minimal, effective method.** Per-row weight norm clipping on decoder layers, with a single hyperparameter (`max_norm`= 2.0), accelerates grokking across optimizers—without weight decay, gradient filtering, or optimizer-specific tuning.
2. **Analysis.** We connect weight norm clipping to four established frameworks: Omnigrok's Goldilocks zone timescale [Liu et al., 2023], the α^L depth scaling law [Vysogorets et al., 2024], Lion's implicit constrained optimization [Chen et al., 2024], and Softmax Collapse prevention [Prieto et al., 2025].
3. **Sign-based optimizers on grokking.** We show that Lion and SignSGD naturally complement norm clipping, with Lion+Clip achieving the fastest convergence across all configurations tested.

2 Method

Weight norm clipping. After each optimizer step, we project every weight row in the decoder layers onto the ℓ_2 ball of radius c:

$$w_{\text{row}} \leftarrow w_{\text{row}} \cdot \min\left(1, \frac{c}{\|w_{\text{row}}\|_2}\right), \tag{1}$$

where c = `max_norm` = 2.0 throughout. This value admits both shrinking (dot products < 1) and amplifying (dot products > 1) interactions between rows, enabling multiplicative dynamics while remaining bounded; $c = 1.0$ restricts dot products to $[-1, 1]$, limiting expressivity. Higher values (e.g., 4.0) weaken regularization. We use $c = 2.0$ as the default and do not tune it per configuration—zero-failure rates across all seeded runs at this value (Tables 1–2) confirm robustness. This is applied only to decoder layer weights (attention projections, MLP layers, and LayerNorm parameters); token embeddings and the output head are not clipped. No weight decay is used.

Hyperparameter trade-off. The method replaces weight decay—a single value requiring schedule tuning—with two decisions: `max_norm` (a constant, 2.0 throughout) and an initialization pattern (`all` for shallow models, `edge_ln` for deep). Neither requires per-run tuning: `max_norm` is fixed across all optimizers, architectures, and learning rates tested; the init pattern is determined by depth alone. In exchange, weight decay and its schedule are eliminated entirely.

`clip_weight_norms` is called after each `optimizer.step()`. `project_to_sphere` is called once at initialization for edge init (Section 4.2). Full implementation at `https://github.com/NiftyliuS/cliptogrok`.

Algorithm 1: Weight Norm Clipping (PyTorch)

```
def clip_weight_norms(model, c = 2.0, skip = {embeddings, head})
    for name, param in model.named_parameters() do
        if name ∉ skip then
            norm ← param.norm(dim= −1, keepdim=True).clamp(min= ϵ);
            param ← param × clamp(norm, max= c) / norm;
def project_to_sphere(model, c = 2.0, patterns = {embeddings, ln_f, head})
    for name, param in model.named_parameters() do
        if name ∈ patterns then
            norm ← param.norm(dim= −1, keepdim=True).clamp(min= ϵ);
            param ← param × c / norm;
```

Edge initialization. For deep models ($L \geq 4$), we additionally project token embeddings, the final LayerNorm, and the output head to norm c before training (`init_pattern='edge_ln'`). Internal decoder layers retain Kaiming initialization. This avoids α^L logit amplification (Section 4.2). The design rationale is asymmetric by necessity:

	Shallow ($L \leq 3$, `all`)		Deep ($L \geq 4$, `edge_ln`)	
Layer type	Init	Clip	Init	Clip
Token embeddings	✓ (to c)	—	✓ (to c)	—
Decoder layers	✓ (to c)	✓ ($\leq c$)	— (Kaiming)	✓ ($\leq c$)
Final LayerNorm	✓ (to c)	✓ ($\leq c$)	✓ (to c)	✓ ($\leq c$)
Output head	✓ (to c)	—	✓ (to c)	—

Decoder layers are clipped every step to prevent memorization shortcuts. Embeddings and head are *not* clipped—cross-entropy requires unconstrained logit magnitudes—but are initialized to c so the network begins at consistent scale. The final LayerNorm (`ln_f`) sits between the last decoder layer and the output head; initializing it to c ensures stable gradient flow at this boundary. In shallow models, initializing *all* layers to c eliminates seed variance entirely (Table 1). In deep models, this inflates residual contributions exponentially (α^L, Section 4.2), so only the boundary layers are touched—hence "edge" initialization.

Experimental setup. We evaluate on modular multiplication ($a \circ b \mod 97$) using a decoder-only transformer. Two sizes: 2-layer (422k params, `dim`= 128, 4 heads) and 8-layer (1.6M params). Training uses 50/50 train/validation split, batch size 512. Convergence criterion: 95% validation accuracy. **All clipped configurations use weight decay = 0.**

Learning rates are depth-dependent. 2-layer: all optimizers use lr= 10^{-3}. 8-layer: Lion lr= 10^{-4} (β_1=0.9, β_2=0.97) [Chen et al., 2023]; Adam and SignSGD lr= 2×10^{-4}. The AdamW baseline (no clipping) uses lr= 10^{-3}, wd= 0.01.

3 Experiments

3.1 Comparison: Baseline vs. Grokfast vs. Clip

Figure 2 compares single-run training dynamics for the three methods on 2-layer modular multiplication, each at its best reported configuration—the same evaluation approach used by Lee et al. [2024]. The baseline (AdamW, wd= 0.01) memorizes fastest (400 steps) but requires 35,040 steps to generalize. Grokfast (Adam + MA filter, window= 100, λ=5.0, wd= 0.01)

reaches 95% validation at 780 steps. Lion+Clip (`max_norm`= 2.0, wd= 0) reaches 95% at 530 steps—a 66× speedup over baseline and 1.5× over Grokfast. The baseline exhibits persistent validation instability post-memorization; both Grokfast and clipping eliminate this.

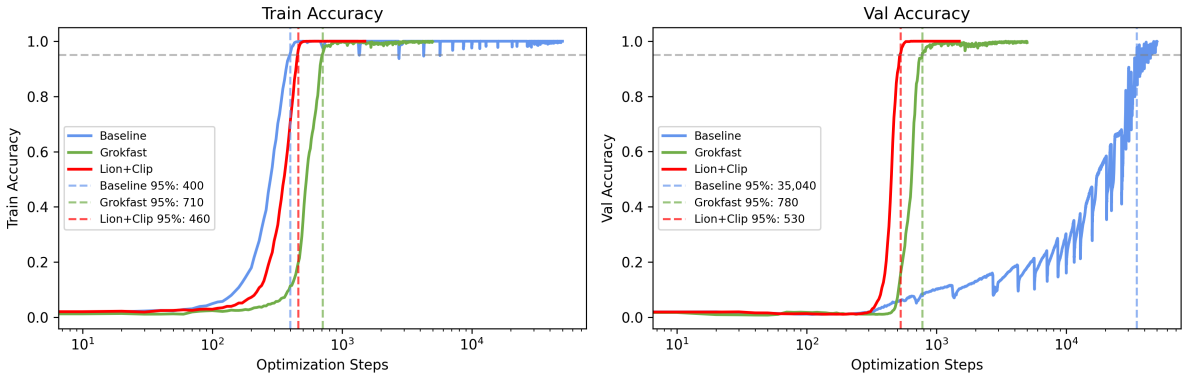


Figure 2: Training and validation accuracy on modular multiplication ($p = 97$, 2-layer), each method at its best reported configuration. Baseline (AdamW, wd= 0.01) reaches 95% val at step 35,040. Grokfast (Adam + MA filter, window= 100, λ=5.0, wd= 0.01) at step 780. Lion+Clip (`max_norm`= 2.0, wd= 0) at step 530—a 66× speedup over baseline and 1.5× over Grokfast. Note: Grokfast also uses a different optimizer (Adam) than the AdamW baseline; our comparison follows the same best-configuration convention.

Decomposing the speedup. The 66× headline uses Lion, not Adam. To separate optimizer and method contributions: Adam+Clip reaches a 2-layer median of 1,200 steps (Table 1)—slower than Grokfast's single-run 780, but with zero seed failures across 200 runs and no gradient history storage. The additional speedup to 550 steps comes from combining clipping with a sign-based optimizer. This decomposition is consistent across scales: on 8-layer models, Adam+Clip alone provides ∼11× speedup over baseline (Table 2), while Lion+Clip reaches 18×.

3.2 2-Layer Seed Stability

Table 1 reports convergence across 100–200 seeds on the 2-layer architecture (all lr= 10^{-3}, wd= 0), sweeping `init_norm` with fixed `max_norm`= 2.0.

Table 1: Steps to 95% val accuracy, 2-layer architecture (422k params). All use lr= 10^{-3}, `max_norm`= 2.0, wd= 0. Budget: 5,000 steps. Failures = seeds not reaching 95% within budget.

Optimizer	init_norm	n	Median [95% CI]	IQR	P95	Fail
Adam	None	100	1350 [1280–1415]	445	3001	10
Adam	1.0	100	1210 [1180–1250]	260	1729	1
Adam	**2.0**	200	**1200 [1160–1245]**	**285**	**1792**	**0**
Lion	None	100	1295 [1170–1515]	865	3279	2
Lion	1.0	100	1260 [1140–1400]	1010	3646	3
Lion	**2.0**	200	**550 [530–560]**	**110**	**780**	**0**
SignSGD	None	100	1510 [1420–1600]	530	2775	1
SignSGD	1.0	100	1460 [1420–1560]	435	2482	0
SignSGD	**2.0**	200	**1140 [1110–1165]**	**180**	**1420**	**0**

At `init_norm`= 2.0, all seeds across all three optimizers converge within 5,000 steps with zero failures. Without normalization, 10% of Adam seeds and isolated Lion/SignSGD seeds

exhibit delayed grokking. Lion achieves the fastest median (550 steps) with the tightest IQR (110), demonstrating particular synergy between sign-based updates and norm clipping.

Figure 3 visualizes the convergence dynamics across all 200 seeds per optimizer at `init_norm`= 2.0.

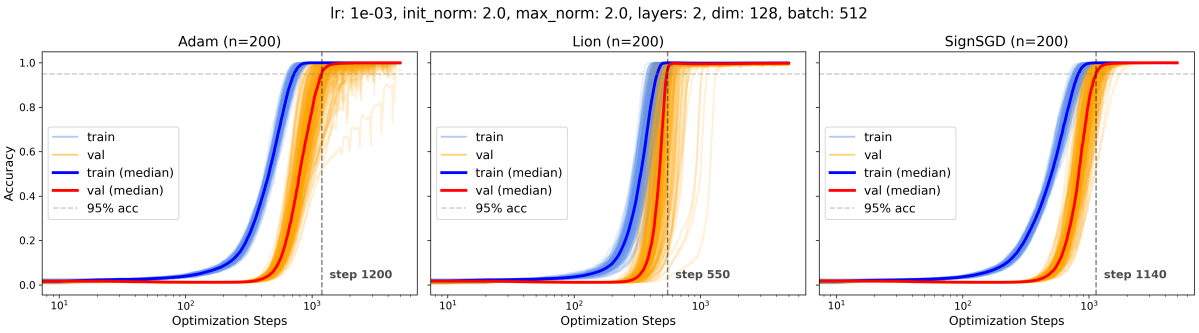


Figure 3: Multi-seed accuracy on the 2-layer architecture (`init_norm`= 2.0, `max_norm`= 2.0, $n = 200$). Dashed lines mark median steps to 95% val accuracy: Adam 1,200 (lr= 10^{-3}), Lion 550 (lr= 10^{-3}), SignSGD 1,140 (lr= 10^{-3}). All seeds converge within 5,000 steps with zero failures.

Adaptive optimizers tolerate imprecise initialization: Adam's per-parameter second moment compensates for scale mismatch, so any normalization eliminates failures. Sign-based optimizers require matched initialization. At `init_norm`= 1.0 with `max_norm`= 2.0, Lion's IQR *worsens* (865 → 1010)—weights must grow from 1.0 toward the clipping boundary before clipping engages, creating a transient regime. At `init_norm`= 2.0, all layers begin at the boundary and clipping engages immediately.

3.3 8-Layer Edge Initialization

8-layer experiments use depth-dependent learning rates: Lion lr= 10^{-4}; Adam and SignSGD lr= 2×10^{-4}. All use wd= 0, `max_norm`= 2.0, budget 50k steps.

Table 2 and Figure 4 present the main result. Edge initialization (`edge_ln`) uniformly improves all metrics across all three optimizers: IQR reduction of 66% (Lion), 72% (Adam), and 61% (SignSGD). Across all 300 edge-init runs, zero seeds failed to reach 95% validation accuracy within 50k steps. Compared to the AdamW baseline (median 28,905 steps), Lion with edge init achieves 18× speedup; Adam ∼ 11×; SignSGD ∼ 10×. Note that cross-optimizer comparisons use different learning rates (Lion 10^{-4}; Adam, SignSGD 2×10^{-4}), each at its validated optimum for this architecture. The baseline IQR of 12,475—43× Lion's 285—shows standard training suffers extreme seed variance.

Scale robustness. At the lower extreme, a 9.8k parameter model (2 layers, dim= 16, 4 heads) fails to train without clipping: AdamW at lr= 10^{-3} does not converge, and lr= 10^{-2} produces unstable grokking around 3,000 steps. With clipping (`max_norm`= 2.0), both Adam and Lion converge cleanly within 2,000 steps at lr= 5×10^{-3}. The method thus spans two orders of magnitude in model size (9.8k–1.6M parameters) without modification.

3.4 Weight Norm Dynamics

Figure 5 shows weight norm evolution during 2-layer training with clipping (`init_norm`= 2.0, `max_norm`= 2.0) for all three optimizers. Decoder norms begin at the clipping threshold and dip during the memorization-to-generalization transition before recovering; unclipped head norms grow freely. Figure 6 shows the baseline (AdamW, no clipping): decoder norms explode to 80×

Table 2: Steps to 95% validation accuracy, 8-layer architecture (1.6M params). All clipped runs: `max_norm`= 2.0, wd= 0. LRs: Lion 10^{-4}; Adam, SignSGD 2×10^{-4}. AdamW baseline: no clipping, wd= 0.01, lr= 10^{-3}. Edge init = `edge_ln` (embeddings + ln_f + head).

Optimizer	Init	n	Median [95% CI]	IQR	P95	Fail
Lion	None	100	2230 [2050–2330]	832	3620	0
Lion	**edge_ln**	100	**1570 [1540–1630]**	**285**	**2430**	**0**
Adam	None	100	3970 [3660–4360]	1480	6747	1
Adam	**edge_ln**	100	**2675 [2620–2765]**	**412**	**3292**	**0**
SignSGD	None	100	4245 [3930–4470]	1838	8288	4
SignSGD	**edge_ln**	100	**2900 [2800–3020]**	**720**	**3955**	**0**
AdamW (no clip)	—	100	28905 [25050–31215]	12475	37197	0

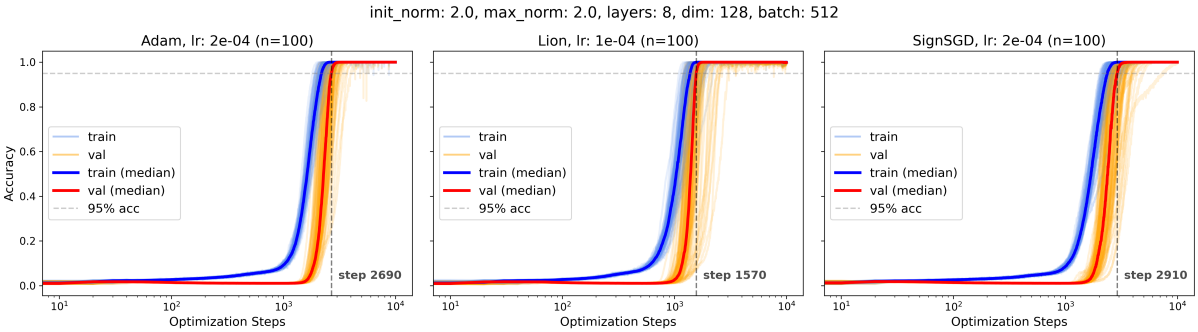


Figure 4: Multi-seed accuracy (8-layer, `edge_ln` init, `max_norm`= 2.0, wd= 0, $n = 100$ per optimizer). Dashed lines: median steps to 95% val acc. Adam: 2,690 (lr= 2×10^{-4}), Lion: 1,570 (lr= 10^{-4}), SignSGD: 2,910 (lr= 2×10^{-4}). All show near-simultaneous train/val convergence with zero failures.

initial values while the model takes 50k steps to generalize with persistent val instability—direct evidence of the unconstrained norm growth that drives delayed grokking.

The pathological dynamics extend to multi-seed statistics. Figure 7 shows the 8-layer AdamW baseline without clipping: validation loss spikes 10–30× above initial values as floating-point errors accumulate (Softmax Collapse), while validation accuracy spreads across two orders of magnitude in convergence time.

3.5 Lion Learning Rate Stability

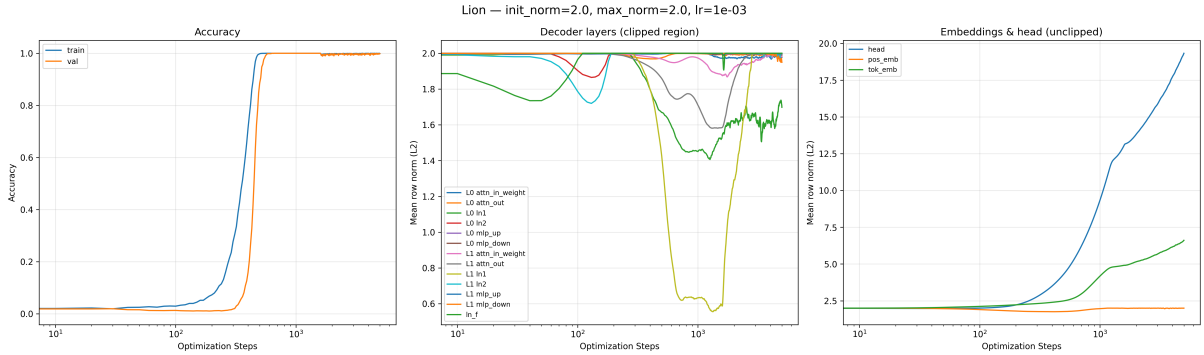
Setup. 20 LRs log-spaced 10^{-4} to 4×10^{-3}, 40 seeds per LR, wd= 0 (apples-to-apples).

Results. Figure 8 presents the main result. With clipping: clean U-curve, optimal LR $\sim 10^{-3}$, median $\sim$530 steps, usable band spanning a full decade. 5 failures out of $\sim$800 runs.

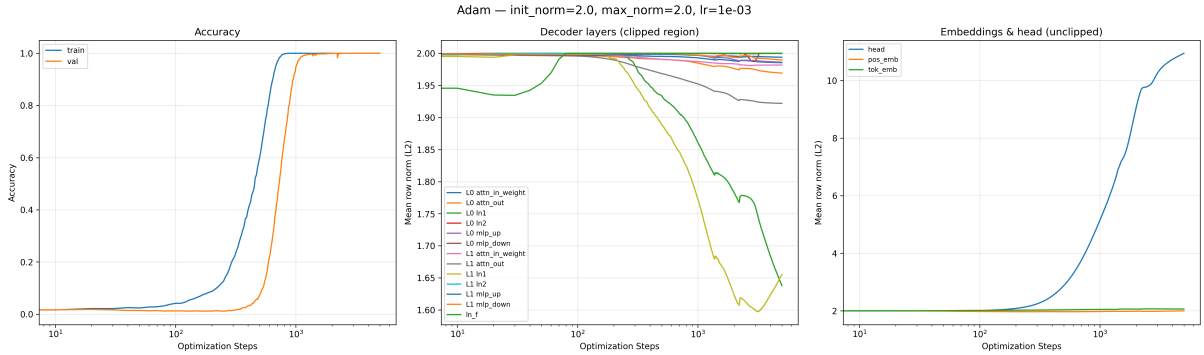
Without clipping: 3–6× slower at *every* LR. At LR= 10^{-4}: 14,000 steps (5.4× slower). Error bars 3–5× wider.

Generalization crispness. Under clipping, the 50/80/95% thresholds are reached within $\sim$200–500 steps of each other (solid lines nearly overlap in Figure 8). Without clipping, the gap spans 4,000–7,000 steps—a prolonged, stochastic transition.

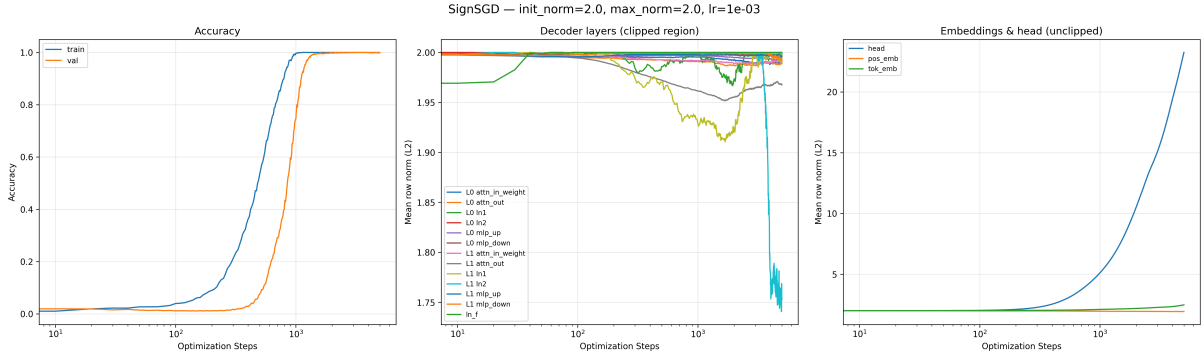
Data-scarce regime. The 25/75 train/validation split is a strictly harder setting: the model sees 25% of the data and must generalize to the remaining 75%. Under this split, Adam+Clip



(a) Lion (lr= 10^{-3}). Decoder norms dip to $\sim$0.6 during transition, recover to boundary. Head grows to $\sim$20.



(b) Adam (lr= 10^{-3}). Norms dip to $\sim$1.6; L1 attn_out and ln1 show largest deviations. Head grows to $\sim$10.



(c) SignSGD (lr= 10^{-3}). Tightest norm control; decoder norms stay within $[1.95, 2.0]$ except brief ln_f dip. Head grows to $\sim$22.

Figure 5: Weight norm evolution (2-layer, `init_norm`= 2.0, `max_norm`= 2.0, wd= 0). Left: accuracy. Center: decoder layer norms (clipped region). Right: embedding/head norms (unclipped). All optimizers show a characteristic norm dip during transition followed by recovery to the clipping boundary.

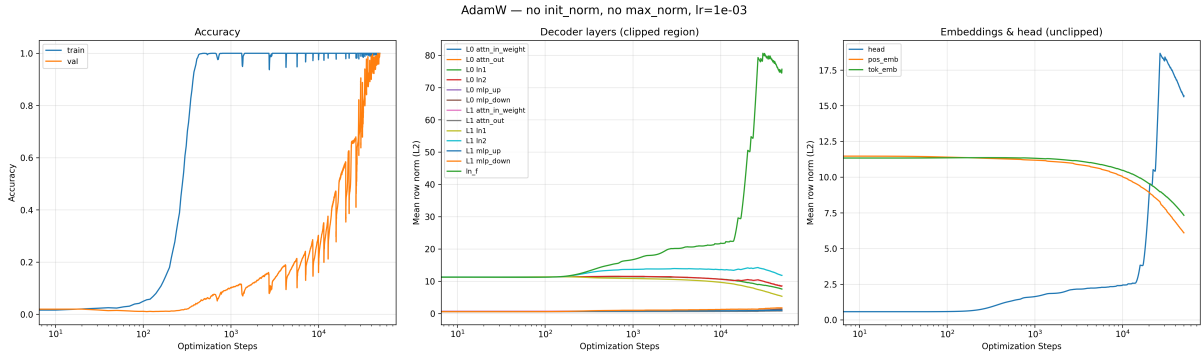
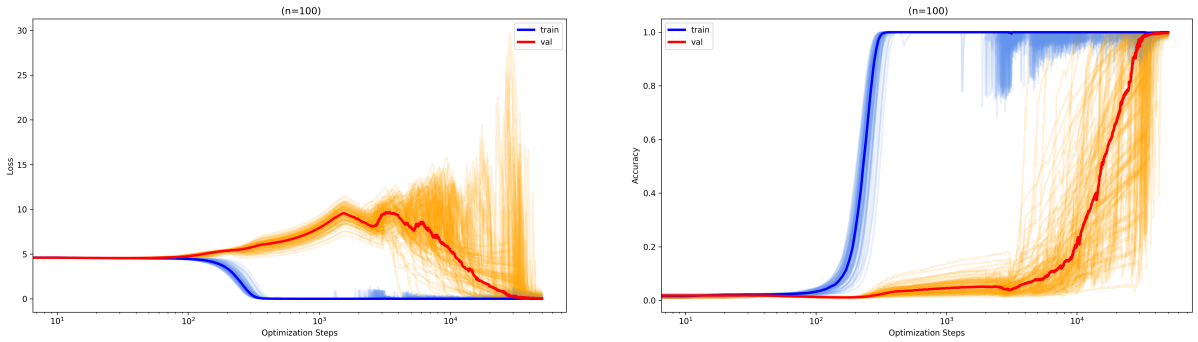


Figure 6: Baseline weight norm dynamics (AdamW, no clipping, lr= 10^{-3}, wd= 0.01, 2-layer). Without norm control, decoder weights (center) explode to 80× initial values. The ln_f norm reaches ∼50 by step 10^4. Training accuracy (left) oscillates through 50k steps with persistent val instability. Compare with clipped training in Figure 5.



(a) Loss (8-layer AdamW, $n = 100$). Median val loss (red) peaks at ∼10 after step ∼10^3; individual seeds spike to 30.

(b) Accuracy ($n = 100$). Train memorizes by step 300; val convergence spreads from 10^3 to 5×10^4 steps.

Figure 7: Softmax Collapse in the AdamW baseline (8-layer, no clipping, wd= 0.01, lr= 10^{-3}, $n = 100$). (a) Validation loss reaches 10–30× initial values—floating-point absorption errors produce erratic gradients. (b) Corresponding accuracy spread. Clipped training (Figure 4) eliminates this pathology entirely.

exhibits loss spikes and SignSGD fails to generalize within budget. Lion+Clip is the only configuration that converges reliably across seeds (Figure 9).

The median convergence (1,530 steps) is roughly 3× slower than the 50/50 split (550 steps), consistent with the reduced training set rather than any failure of the method. That Lion alone succeeds here reinforces the synergy between sign-based updates with momentum and norm clipping: bounded weights prevent memorization shortcuts, while momentum provides the gradient signal averaging that compensates for the smaller training set.

Post-convergence oscillation. After reaching 95%+ validation accuracy, Lion can exhibit minor val oscillations driven by unbounded head norm growth. Optionally clipping the output head to a loose bound (e.g., 10.0) suppresses this without affecting convergence speed. We do not include head clipping in the default method because it is Lion-specific—Adam's second-moment statistics conflict with external norm constraints on the head, producing worse oscillations rather than better.

Interpretation. Clipping decouples LR from escape risk. LR controls exploration speed *within* the bounded weight set, not the risk of norm explosion—explaining the 3–6× uniform speedup.

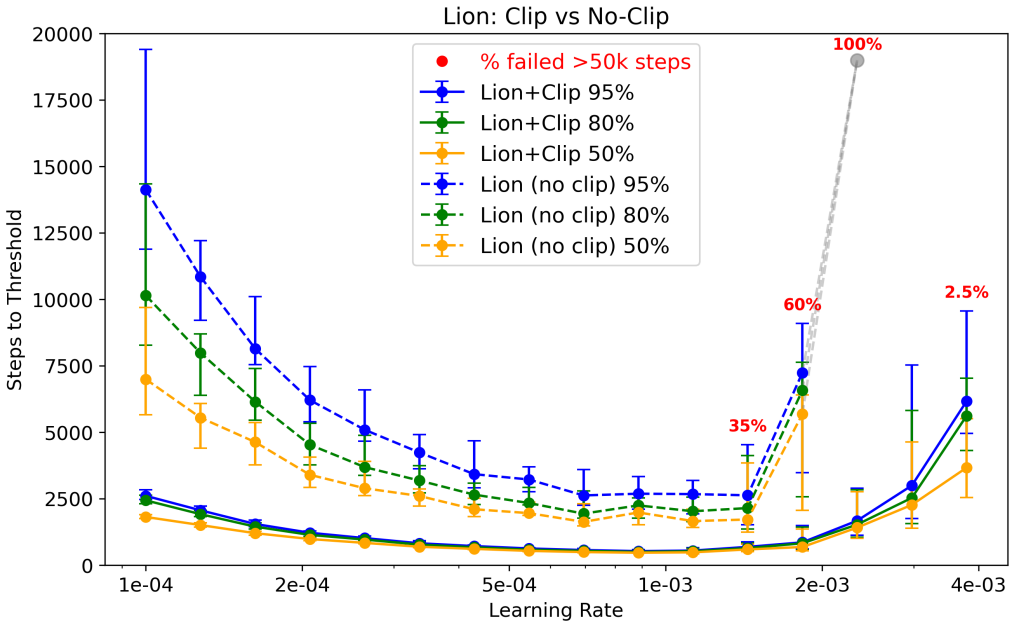


Figure 8: LR tolerance: Lion+Clip (solid) vs. Lion no-clip (dashed), 40 seeds/LR, wd= 0. Three accuracy thresholds shown (50%, 80%, 95%). Clipping provides 3–6× speedup at every LR with compressed variance. Red percentages and gray dots indicate no-clip failure rates at high LR (35%, 60%, 100%). Clipped runs show only 2.5% failure at LR= 4×10^{-3}.

Section 4.3 provides the theoretical basis.

4 Analysis: Why Weight Norm Clipping Works

We analyze why weight norm clipping accelerates generalization through four complementary lenses. Each established framework independently predicts that bounding weight norms should help; clipping implements this directly.

4.1 Collapsing the Grokking Timescale

Liu et al. [2023] introduced the Goldilocks zone: a spherical shell in weight space at norm $\approx w_c$ where generalizing solutions concentrate. The grokking timescale depends on how far initialization w_0 lies from w_c:

$$t_{\text{grok}} \approx \frac{1}{\gamma} \ln\left(\frac{w_0}{w_c}\right), \tag{2}$$

where γ is the weight decay rate. Lyu et al. [2024] formalized this for homogeneous networks, proving transition from kernel regime (memorization) to max-margin regime (generalization) at $t \sim \frac{1}{\lambda} \ln \alpha$.

Weight norm clipping eliminates this delay by constraining $\|w\| \leq c$ at every step. If $c \approx w_c$, then $\ln(w_0/w_c) \to 0$ and the grokking timescale collapses regardless of γ. Two consequences: (1) **weight decay becomes unnecessary**—clipping keeps norms in the zone from the start; (2) **robustness to exact** c—the method works with `max_norm`= 2.0 across all tested configurations without tuning, suggesting the Goldilocks zone is broad relative to the clipping threshold.

Proposition 1 (Timescale collapse under clipping). *Let w_c be the Goldilocks zone radius from Liu et al. [2023] and suppose weights are projected to $\|w\| \leq c$ after every step. If $c \leq w_c$, then the grokking timescale satisfies $t_{\text{grok}} = O(1)$ independent of initialization w_0 and weight*

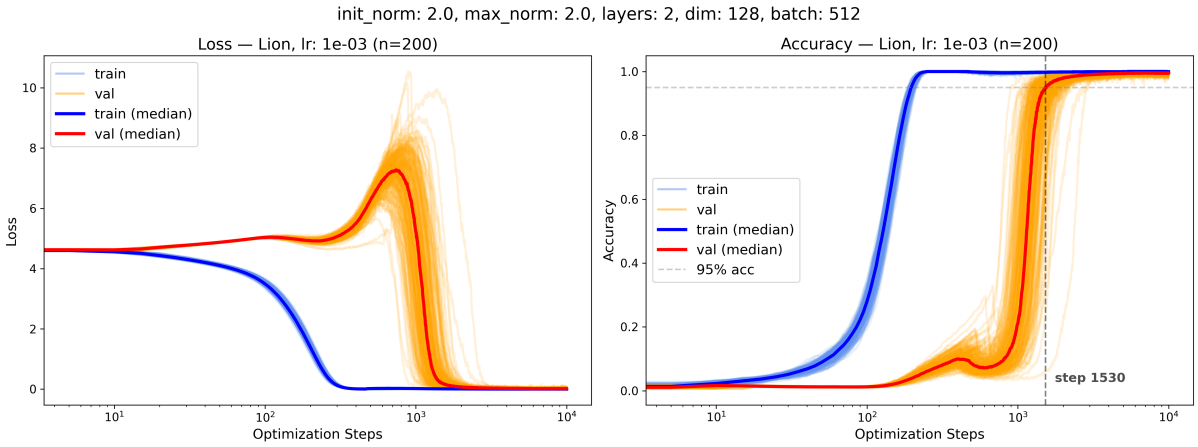


Figure 9: Multi-seed accuracy with 25/75 train/val split (2-layer, Lion, lr= 10^{-3}, `init_norm`= 2.0, `max_norm`= 2.0, wd= 0, $n = 200$). Dashed line: median 95% val accuracy at step 1,530. All seeds converge within 10,000 steps. Adam and SignSGD struggle under this split (not shown); Lion's momentum provides the directional consistency needed when fewer training examples are available per step.

decay rate γ. This follows directly from Eq. (2): clipping removes the dependence on w_0/w_c by enforcing $w \leq w_c$ at every step, collapsing the logarithmic term.

Remark 1. *Clipping is strictly stronger than weight decay for norm control. Weight decay applies $w \leftarrow (1 - \lambda\eta)\, w$, requiring many steps. Clipping enforces $\|w\| \leq c$ in a single projection.*

4.2 Depth Amplification and the α^L Scaling Law

Vysogorets et al. [2024] showed that for L-layer homogeneous networks, scaling weights by α amplifies output by α^L.

Motivating calculation (homogeneous case). For a purely homogeneous L-layer network (no residual connections, no LayerNorm), scaling all weights to `init_norm`= c inflates each layer by $\alpha = c/\|w_{\text{init}}\|$:

$$\text{Output scale} \;\propto\; \left(\frac{c}{\|w_{\text{init}}\|}\right)^L. \tag{3}$$

For $c = 2.0$, $\|w_{\text{init}}\| \approx 0.6$ (Kaiming, dim= 128): at $L = 2$, $\alpha^L \approx 11$; at $L = 8$, $\alpha^L \approx 13{,}000$.

Our transformer uses residual connections and LayerNorm, so this homogeneous analysis does not directly apply. However, it motivates why `init_norm`= 2.0 on all layers improves 2-layer but harms 8-layer models. In residual networks, skip connections convert the exponential α^L risk into bounded linear perturbation $L \cdot c$:

$$h_L = h_0 + \sum_{\ell=0}^{L-1} f_\ell(h_\ell), \tag{4}$$

explaining why the *same* `max_norm`= 2.0 works across depths with edge initialization. Table 2 validates this: edge init achieves zero failures across all 300 runs on the 8-layer architecture.

4.3 Sign-Based Optimizers and Norm Clipping

Lion as constrained optimization. Chen et al. [2024] proved Lion with weight decay λ solves $\min_x f(x)$ s.t. $\|x\|_\infty \leq 1/\lambda$ via Frank–Wolfe [Sfyraki & Wang, 2025]. In our experiments, we use wd= 0—so this ℓ_∞ constraint is absent and clipping provides the *only* norm control.

Why sign-based optimizers benefit from clipping. Lion's update is $\text{sign}(m_t)$: each parameter changes by exactly $\pm\text{lr}$ per step, regardless of gradient magnitude. Without norm control, this uniform step size means weight norms grow linearly in training steps. Clipping caps this growth, preventing the slow drift toward Softmax Collapse. The combination—bounded updates from the sign operation, bounded norms from clipping—keeps optimization stable across a wide LR range. Figure 8 confirms 3–6× speedup at every tested LR with 0% failures up to LR$= 2 \times 10^{-3}$, versus 100% failure without clipping at the same LR.

Observation: Grokfast + Adam → approximately sign-based updates. Grokfast amplifies slow-varying gradient components via an EMA filter, then feeds the modified gradient into Adam:

$$\text{Grokfast:} \quad g_t^{\text{slow}} = \beta\, g_{t-1}^{\text{slow}} + (1-\beta)\,\nabla f_t, \quad \tilde{g}_t = \nabla f_t + \lambda\, g_t^{\text{slow}} \tag{5}$$

$$\text{Adam step:} \quad \hat{m}_t/\sqrt{\hat{v}_t} \;\approx\; \text{sign}(\hat{m}_t) \quad \text{when } \hat{v}_t \approx \hat{m}_t^2 \tag{6}$$

Adam's second-moment normalization divides each parameter's update by its own RMS, driving updates toward ± 1. This effect is amplified when Grokfast's EMA filter smooths the gradient signal (reducing variance within the second-moment estimate). The resulting effective update—sign of a momentum-weighted gradient—resembles Lion's update rule:

$$\text{Lion:} \quad u_t = \text{sign}(\beta_2\, m_{t-1} + (1-\beta_2)\,\nabla f_t) \tag{7}$$

Put differently: Grokfast applies momentum at the *gradient* level; Lion applies it at the *optimizer* level. Both produce sign-based, directionally consistent updates. This is an informal observation, not a proof—it depends on Adam's moment estimates being well-conditioned and Grokfast's λ being large enough to dominate the raw gradient. We note it because it may explain why both methods achieve comparable speedups despite different mechanisms, and because it suggests that the grokking acceleration literature may have been converging on sign-based dynamics from multiple directions independently.

4.4 Preventing Softmax Collapse

Prieto et al. [2025] identified Softmax Collapse (SC): after memorization, the training objective (negative log-likelihood) can still decrease by increasing logit magnitudes, which requires growing weight norms. This post-memorization weight growth drives logit amplification until float32 overflow causes absorption errors in softmax, producing erratic gradients that stall generalization.

Clipping eliminates this cascade by bounding decoder norms: $\|w_{\text{row}}\|_2 \leq c$ directly caps logit growth proportional to c, preventing overflow regardless of training duration. The weight norm dynamics confirm this: without clipping, decoder norms reach 80× initial values (Figure 6); with clipping, norms stay bounded and SC never occurs (Figure 5).

Remark 2 (Weight decay vs. clipping for SC)**.** *Weight decay prevents SC* eventually*; clipping prevents it* immediately*—eliminating the plateau-then-grokking pattern.*

5 Related Work

The grokking phenomenon. Power et al. [2022] first observed delayed generalization. Nanda et al. [2023] provided mechanistic interpretability, identifying Fourier multiplication circuits and three training phases.

Weight norm and the Goldilocks zone. Liu et al. [2023] showed grokking occurs when initialized outside the zone. Lyu et al. [2024] proved the kernel-to-max-margin transition at $t \sim (1/\lambda)\ln\alpha$. Our method operationalizes these insights via per-row clipping.

Depth and logit scaling. Vysogorets et al. [2024] connected weight scale to α^L logit amplification. Fan et al. [2024] examined grokking vs. depth but didn't propose norm control.

Numerical stability. Prieto et al. [2025] identified Softmax Collapse. Their $\perp$Grad modifies gradients; we prevent weight growth directly.

Gradient-based acceleration. Lee et al. [2024] proposed Grokfast ($> 50\times$ speedup). We observe that its EMA filter, combined with Adam's normalization, produces approximately sign-based updates (Section 4.3). NeuralGrok [Zhou et al., 2025] learns a neural gradient filter but doesn't transfer.

Normalized representations. nGPT [Loshchilov et al., 2025] normalizes all vectors to unit norm for 4–20$\times$ LM speedup. The connection between our weight-space clipping and their representation-space normalization warrants investigation. Spectral normalization [Miyato et al., 2018] constrains the operator norm of each weight matrix (largest singular value), controlling worst-case input amplification. This is a complementary constraint: spectral norm bounds how much a layer can amplify any input direction, while per-row clipping bounds individual neuron magnitudes. The two are not interchangeable—a matrix can satisfy one constraint while violating the other. GrokTransfer [Xu et al., 2025] uses embedding transfer (46-step gap).

Sign-based optimization. Lion [Chen et al., 2023] solves ℓ_∞-constrained optimization [Chen et al., 2024]. Sfyraki & Wang [2025] unified Lion/Muon under Frank–Wolfe. Tveit et al. [2025] showed Muon accelerates grokking via spectral norm control.

6 Conclusion

Weight norm clipping accelerates grokking by 66$\times$ on 2-layer models and 18$\times$ on 8-layer models (1.6M parameters), with no additional memory overhead. Four established frameworks—Omnigrok timescale collapse, α^L depth scaling, constrained optimization, and SC prevention—each independently predict this outcome. Edge initialization eliminates all failures across 300 runs and reduces IQR by 61–72%. The method spans two orders of magnitude in model size (9.8k–1.6M parameters) without modification.

The near-simultaneous train/val convergence suggests the traditional memorization-generalization gap may be an artifact of unconstrained optimization. Lion+Clip emerges as the recommended configuration: fastest convergence, tightest seed variance, widest LR tolerance, and the only optimizer that generalizes reliably under data-scarce conditions (25/75 split).

Limitations. All experiments use modular arithmetic ($a \times b \mod 97$). While results hold across model scales (9.8k–1.6M parameters), depths (2–8 layers), optimizers, and training configurations, we have not validated on natural language or other domains. Modular arithmetic is the standard benchmark for grokking research, but the practical value of accelerated grokking depends on whether the same weight norm dynamics govern generalization in larger-scale settings.

Future work. The most direct extension is language model pretraining with norm clipping, particularly in combination with nGPT's representation-space normalization [Loshchilov et al., 2025]. If post-step clipping can replace weight decay scheduling at scale—as it replaces weight decay entirely in our experiments—the practical impact would be substantial. The method's compatibility with sign-based optimizers (Lion, SignSGD) is especially relevant given their lower memory footprint for large models.

Reproducibility. Code and experiment scripts are available at `https://github.com/NiftyliuS/cliptogrok`.

References

Chen, X., Liang, C., Huang, D., Real, E., Wang, K., Liu, H., Pham, H., Dong, X., Luber, T., Cho, J., Luo, L., and Le, Q. V. Symbolic discovery of optimization algorithms. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Chen, X., Liang, C., Huang, D., Real, E., Pham, H., Dong, X., Luber, T., Cho, J., Luo, L., and Le, Q. V. Lion secretly solves constrained optimization: As Lyapunov predicts. In *International Conference on Learning Representations (ICLR)*, 2024.

Fan, Z., Liu, Z., and Wang, Z. Deep grokking: Would deep neural networks generalize better? *arXiv preprint arXiv:2405.19454*, 2024.

Lee, J., Park, N., Lee, J., and Shin, J. Grokfast: Accelerated grokking by amplifying slow gradients. *arXiv preprint arXiv:2405.20233*, 2024.

Liu, Z., Michaud, E. J., and Tegmark, M. Omnigrok: Grokking beyond algorithmic data. In *International Conference on Learning Representations (ICLR)*, 2023.

Loshchilov, I., Hsieh, C.-P., Sun, S., and Ginsburg, B. nGPT: Normalized transformer with representation learning on the hypersphere. In *International Conference on Learning Representations (ICLR)*, 2025.

Lyu, K., Jin, J., Li, Z., Du, S. S., Lee, J. D., and Hu, W. Dichotomy of early and late phase implicit biases can provably induce grokking. In *International Conference on Learning Representations (ICLR)*, 2024.

Takeru Miyato, Toshiki Kataoka, Masanori Koyama, and Yuichi Yoshida. Spectral normalization for generative adversarial networks. In *International Conference on Learning Representations*, 2018.

Nanda, N., Chan, L., Lieberum, T., Smith, J., and Steinhardt, J. Progress measures for grokking via mechanistic interpretability. In *International Conference on Learning Representations (ICLR)*, 2023.

Power, A., Burda, Y., Edwards, H., Babuschkin, I., and Misra, V. Grokking: Generalization beyond overfitting on small algorithmic datasets. In *ICLR Workshop on Mathematics of Modern Machine Learning*, 2022.

Prieto, L., Barsbey, M., Vasilakis, C., and Kaski, S. Grokking at the edge of numerical stability. In *International Conference on Learning Representations (ICLR)*, 2025.

Sfyraki, M. and Wang, Z. Lions and muons: Optimization via stochastic Frank–Wolfe. *arXiv preprint arXiv:2506.04192*, 2025.

Tveit, T. A., Tveit, A., and Breck, E. Muon optimizer accelerates grokking. *arXiv preprint arXiv:2504.16041*, 2025.

Vysogorets, A., Dawid, A., and Kempe, J. Deconstructing the Goldilocks zone of neural network initialization. In *International Conference on Machine Learning (ICML)*, 2024.

Xu, Z., et al. Let me grok for you: Accelerating grokking via embedding transfer from a weaker model. In *International Conference on Learning Representations (ICLR)*, 2025.

Zhou, Y., et al. NeuralGrok: Accelerate grokking by neural gradient transformation. *arXiv preprint arXiv:2504.17243*, 2025.